Lab 2 Report

Point-to-Point Communication in MPI

Name: Ahmad Bin Rashid

Roll No: 2023-CS-53

Instructor: Sir Waqas Ali

Course: Distributed and Parallel Computing

University of Engineering and Technology

March 3, 2026

1 Introduction

The objective of this lab is to Understand how MPI processes execute the same binary but differentiate their work using rank.

2 Tools and Environment

- Programming Language: C
- Parallel Framework: MPI (OpenMPI)
- Compiler: `mpicc`
- Execution Tool: `mpirun`
- Operating System: Linux

3 Programs Implemented

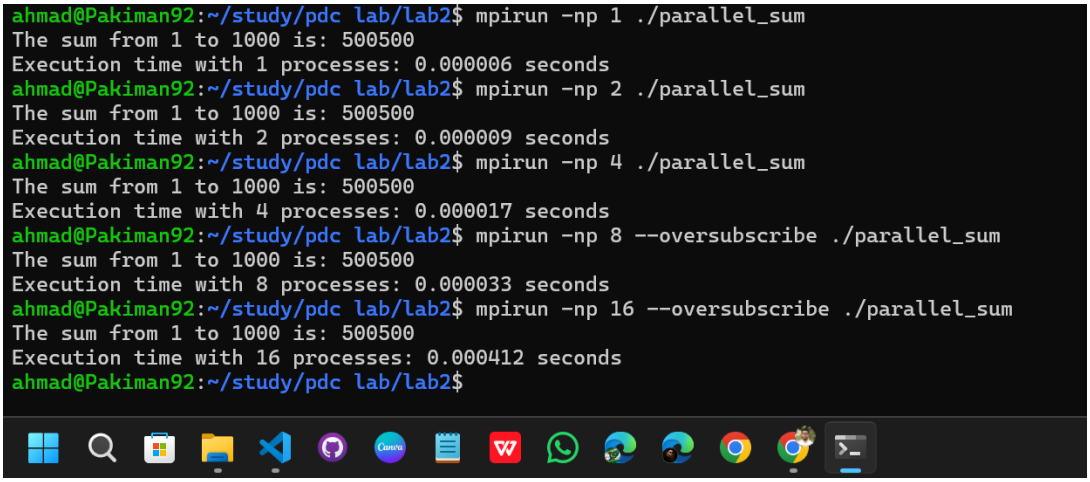
- Parallel Sum
- Exercise 2 program
- Exercise 3 Program

4 Performance Analysis of `Exercise 1`

4.1 Execution Time and Speedup

Processes	**Execution Time (s)**	**Speedup**
1	0.000006	1.00
2	0.000009	0.67
4	0.000017	0.35
8	0.000033	0.18
16	0.000412	0.01

Table 1: Execution Time and Speedup for Parallel Sum



```
ahmad@Pakiman92:~/study/pdc lab/lab2$ mpirun -np 1 ./parallel_sum
The sum from 1 to 1000 is: 500500
Execution time with 1 processes: 0.000006 seconds
ahmad@Pakiman92:~/study/pdc lab/lab2$ mpirun -np 2 ./parallel_sum
The sum from 1 to 1000 is: 500500
Execution time with 2 processes: 0.000009 seconds
ahmad@Pakiman92:~/study/pdc lab/lab2$ mpirun -np 4 ./parallel_sum
The sum from 1 to 1000 is: 500500
Execution time with 4 processes: 0.000017 seconds
ahmad@Pakiman92:~/study/pdc lab/lab2$ mpirun -np 8 --oversubscribe ./parallel_sum
The sum from 1 to 1000 is: 500500
Execution time with 8 processes: 0.000033 seconds
ahmad@Pakiman92:~/study/pdc lab/lab2$ mpirun -np 16 --oversubscribe ./parallel_sum
The sum from 1 to 1000 is: 500500
Execution time with 16 processes: 0.000412 seconds
ahmad@Pakiman92:~/study/pdc lab/lab2$
```

Figure 1: Parallel Sum Results

4.2 Speedup Plot

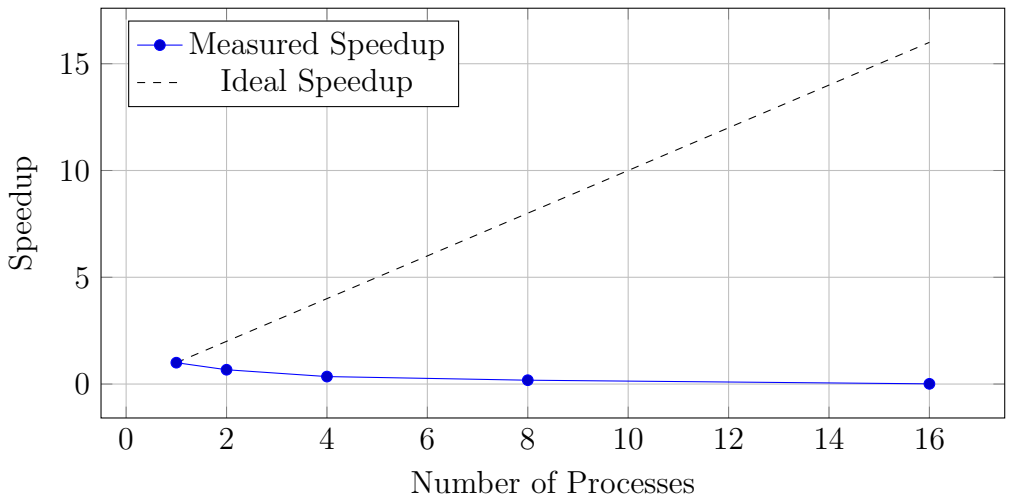


Figure 2: Measured vs Ideal Speedup for Parallel Sum

4.3 Discussion

The speedup decreases as the number of processes increases because the problem size is extremely small, making communication overhead dominate computation time. According to Amdahl's Law, the sequential fraction of the program limits scalability, and in this case, the communication cost is larger than the parallel work. Additionally, process startup overhead and load imbalance contribute to poor efficiency, especially at higher process counts.

5 Analysis of Exercise 2

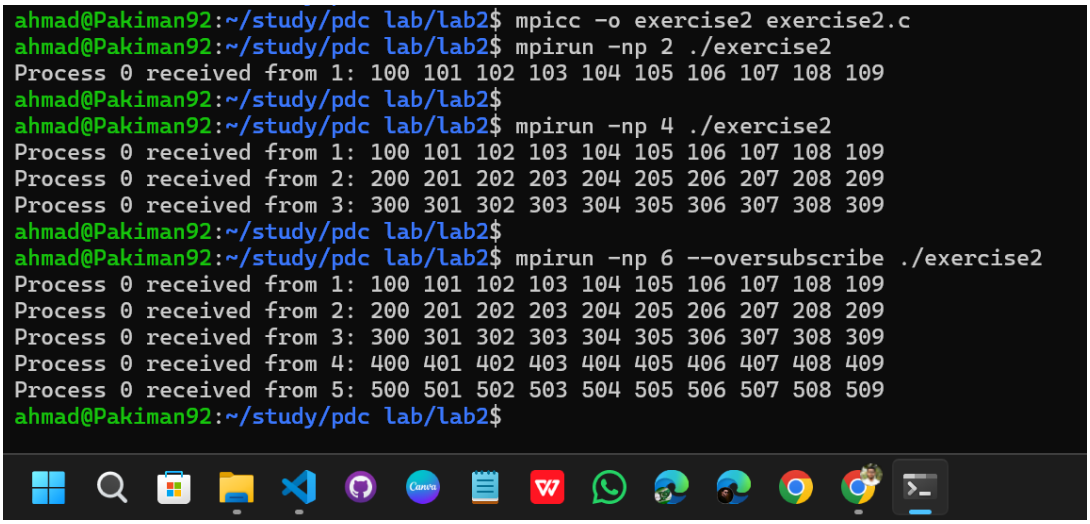


Figure 3: Exercise 2 Program Results

5.1 Source Code

```c
#include <stdio.h>
#include <mpi.h>

#define ARRAY_SIZE 10

int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank != 0) {
        // Worker: generate array and send to process 0
        int data[ARRAY_SIZE];
        for (int i = 0; i < ARRAY_SIZE; i++) {
            data[i] = rank * 100 + i;
        }
        MPI_Send(data, ARRAY_SIZE, MPI_INT, 0, rank,
           MPI_COMM_WORLD);
    } else {
        // Master: receive arrays from all workers and print them
        int recv_buf[ARRAY_SIZE];

        for (int j = 1; j < size; j++) {
            MPI_Recv(recv_buf, ARRAY_SIZE, MPI_INT, j, j,
               MPI_COMM_WORLD, &status);
            printf("Process 0 received from %d:", j);
```

```

            for (int i = 0; i < ARRAY_SIZE; i++) {
                printf(" %d", recv_buf[i]);
            }
            printf("\n");
        }
    }

    MPI_Finalize();
    return 0;
}
```

5.2 Analysis

This program demonstrates point-to-point communication in MPI using `MPI_Send` and `MPI_Recv`. Each worker process generates its own data and sends it to the master process, illustrating the distributed-memory communication model.

6 Analysis of Exercise 3

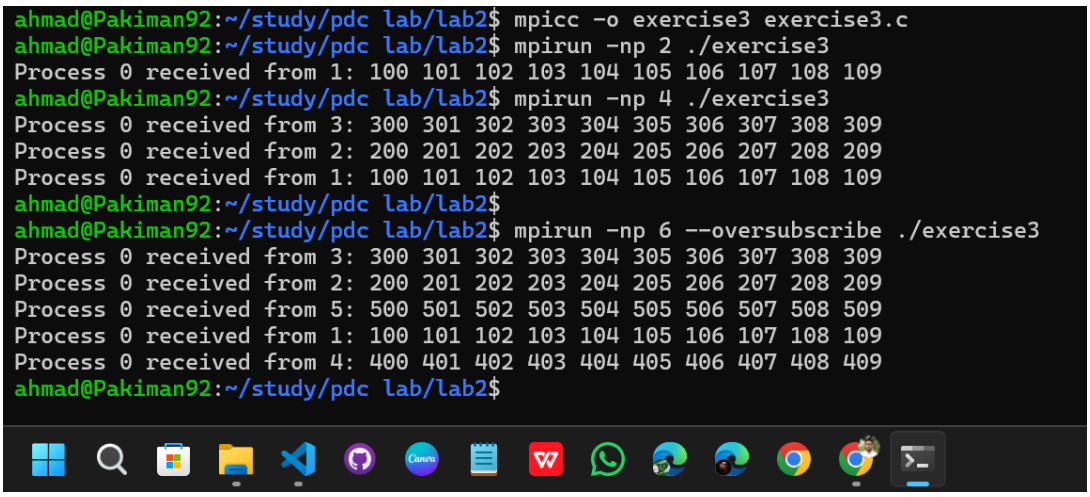


Figure 4: Exercise 3 Program Results

6.1 Source Code

```
#include <stdio.h>
#include <mpi.h>

#define ARRAY_SIZE 10

int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Status status;

```

```c
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank != 0) {
        // Worker: generate array and send to process 0
        int data[ARRAY_SIZE];
        for (int i = 0; i < ARRAY_SIZE; i++) {
            data[i] = rank * 100 + i;
        }
        MPI_Send(data, ARRAY_SIZE, MPI_INT, 0, rank,
           MPI_COMM_WORLD);
    } else {
        // Master: receive arrays from all workers and print them
        int recv_buf[ARRAY_SIZE];

        for (int j = 1; j < size; j++) {

            MPI_Recv(recv_buf, ARRAY_SIZE, MPI_INT,
               MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &
               status);
            int source = status.MPI_SOURCE;
            printf("Process 0 received from %d:", source);

            for (int i = 0; i < ARRAY_SIZE; i++) {
                printf(" %d", recv_buf[i]);
            }
            printf("\n");
        }
    }

    MPI_Finalize();
    return 0;
}
```

6.2 Analysis

This version demonstrates flexible message reception using `MPI_ANY_SOURCE` and `MPI_ANY_TAG`. The master process can receive messages in any order, making the program more dynamic and better suited for unpredictable communication patterns.

7 Reflection on Point-to-Point vs Collective Communication

Point-to-point communication requires explicit coordination between sender and receiver, making programs more complex and error-prone. The programmer must carefully manage message order, tags, and potential deadlocks, which increases development difficulty. In contrast, collective operations such as `MPI_Reduce` abstract these details and provide optimized, synchronized communication patterns. Collective operations are generally more scalable and easier to implement, while point-to-point communication offers greater flexibility but at the cost of higher programming complexity.